



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

## **Lab experiment-1**

### **8-Puzzle Problem**

#### **Aim**

To solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm.

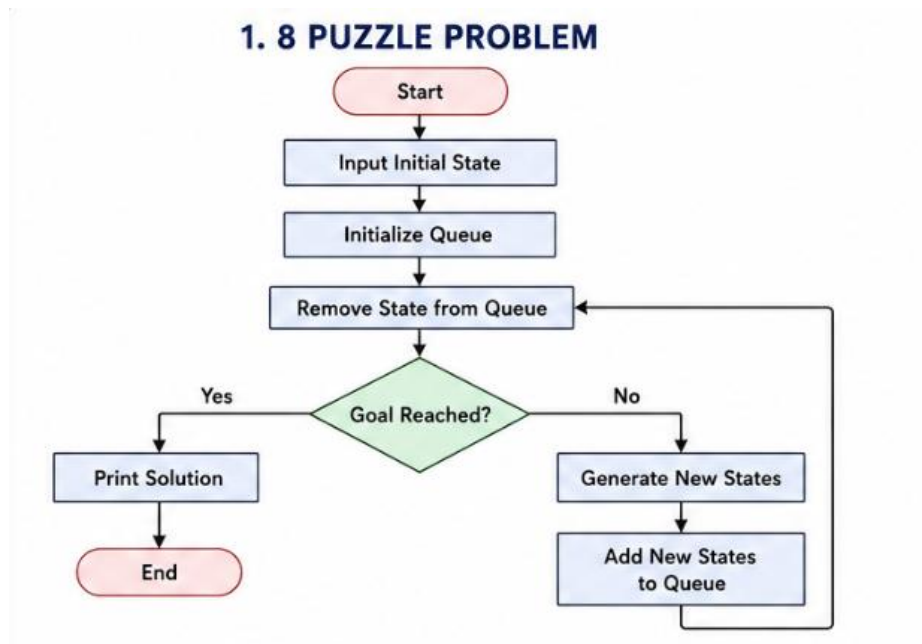
#### **Objective**

To find the shortest path from the initial state to the goal state by moving the blank tile.

#### **Algorithm**

1. Start with the initial puzzle state.
2. Insert the state into a queue.
3. Remove the first state from the queue.
4. Check whether it is the goal state.
5. If not, generate all possible next states.
6. Add unvisited states to the queue.
7. Repeat until the goal state is reached.

## Flowchart



## Code

```
from collections import deque
```

```
goal = [1, 2, 3,
```

```
        4, 5, 6,
```

```
        7, 8, 0]
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    index = state.index(0)
```

```
    moves = {
```

```
        0: [1, 3],
```

```
        1: [0, 2, 4],
```

```
        2: [1, 5],
```

```
        3: [0, 4, 6],
```

```
        4: [1, 3, 5, 7],
```

```

    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}

for move in moves[index]:
    new_state = state[:]
    new_state[index], new_state[move] = new_state[move], new_state[index]
    neighbors.append(new_state)

return neighbors

def solve(start):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.append(state)
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None

start = [1, 2, 3, 4, 5, 6,
        0, 7, 8]

```

```

solution = solve(start)

if solution:

    print("Solution Steps:")

    for step in solution:

        print(step[0:3])

        print(step[3:6])

        print(step[6:9])

        print()

else:

    print("No Solution Found")

```

## Output

The screenshot shows a Python IDE with two windows. The left window displays the source code for '8puzzels.py', which implements a Breadth-First Search (BFS) algorithm to solve the 8-puzzle problem. The code defines a goal state, a function to get neighbors, and a solve function that uses a queue and a visited set. The right window shows the output of the program, which prints the solution steps as a sequence of 3x3 grids representing the state of the puzzle at each step.

```

8puzzels.py - C:/Users/shrut/AppData/Local/Programs/Python/Python314/8puzzels.py (3.14.2)
File Edit Format Run Options Window Help
from collections import deque
goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]
def get_neighbors(state):
    neighbors = []
    index = state.index(0)
    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }
    for move in moves[index]:
        new_state = state[:]
        new_state[index], new_state[move] = new_state[move], new_state[index]
        neighbors.append(new_state)
    return neighbors
def solve(start):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.append(state)
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None
start = [1, 2, 3,
        4, 5, 6,
        0, 7, 8]
solution = solve(start)
if solution:
    print("Solution Steps:")

```

```

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:1
on win32
Enter "help" below or click "Help" above for more info
>>>
=====
ocal/Programs/Python/Python314/8puzzels.py =====
=====
Solution Steps:
[1, 2, 3]
[4, 5, 6]
[0, 7, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>

```

## Result

The 8-Puzzle problem is solved successfully using BFS.

## Conclusion

The BFS algorithm finds the shortest path to reach the goal state.